

Customer Churn Prediction System

Technical Report · v2.0

Generated: July 2026

Customer Churn Prediction System

An End-to-End Machine Learning Pipeline for Telecom Subscriber Retention

Project Technical Report

Version 2.0

ABSTRACT

Customer churn · the loss of subscribers to a competing service · is one of the most costly and persistent problems facing the telecommunications industry, where acquiring a new customer can cost several times more than retaining an existing one. This project delivers a complete, production-oriented machine learning system that predicts the likelihood of an individual customer churning, enabling proactive retention intervention before the customer actually leaves.

The system is built on the publicly available Telco Customer Churn dataset comprising 7,043 customer records with a 26.5% churn rate. The work spans the full data science lifecycle: exploratory data analysis, data cleaning, feature engineering, model training and hyperparameter tuning, model explainability, model persistence, a REST API service, and an interactive web interface. Three candidate algorithms · Logistic Regression, Random Forest, and XGBoost · were trained and evaluated, with a tuned Random Forest selected as the production model on the basis of its ROC-AUC score.

Beyond the base thirty encoded attributes, fourteen engineered features were introduced · including charge-per-month ratios, service-count aggregates, and high-risk interaction flags such as fiber-optic-without-technical-support · raising the feature space to forty-four inputs and improving discriminative power to a ROC-AUC of 0.846. The final model is served through a FastAPI application exposing a validated /predict endpoint, and is consumed through a Bootstrap web form that returns a churn decision and calibrated probability in real time. The report documents the architecture, pipeline, per-phase design decisions, and the explicit rationale for each technology choice against its alternatives.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

This chapter establishes the context for the project. It introduces the churn problem within the telecommunications domain, states the aim and measurable objectives, defines the problem precisely, describes the proposed system and its scope, and closes with the assumptions, constraints, social benefits, and a layout of the remaining report.

1.1 Introduction

The telecommunications sector operates in a saturated, highly competitive market where subscriber growth is largely a zero-sum game · one provider's gain is another's loss. In this environment, the ability to keep existing customers is a stronger driver of profitability than the ability to acquire new ones, because acquisition involves marketing spend, onboarding cost, and promotional discounts, whereas retention primarily requires timely, targeted engagement. Industry analyses consistently place the cost of acquiring a new customer at five to seven times the cost of retaining an existing one.

Churn is the event in which a customer discontinues their subscription. Because churn is often preceded by observable behavioural and contractual signals · short tenure, month-to-month contracts, high monthly charges, absence of value-added services · it is a problem well suited to supervised machine learning. If a provider can reliably score each customer for churn risk, it can direct its finite retention budget at the customers most likely to leave, rather than spending it uniformly or reactively.

This project moves from that general problem to a concrete, working solution: a machine learning system that ingests a customer's demographic, service, and account attributes and returns both a binary churn decision and a probability score. The system is engineered not merely as a notebook experiment but as a deployable service with input validation, a REST API, and a user-facing web interface, reflecting the end-to-end nature of real

machine learning products.

1.2 Aim & Objectives

Aim: To design, train, and deploy a machine learning system that accurately predicts telecom customer churn and exposes those predictions through a validated API and an interactive web interface.

Objectives:

To perform exploratory data analysis on the Telco Customer Churn dataset and characterise the distribution of churn against customer attributes.

To clean the raw data, resolve missing and malformed values, and remove duplicate records.

To engineer domain-informed features that improve the predictive signal beyond the raw attributes.

To train and compare at least three classification algorithms and select the best performer on ROC-AUC.

To tune the selected model's hyperparameters using cross-validated grid search.

To interpret the model's behaviour using SHAP to ensure predictions are explainable.

To persist the trained model and serve it through a FastAPI REST endpoint with strict input validation.

To build a lightweight web interface that allows a non-technical user to obtain a churn prediction.

1.3 Problem Statement

Telecom providers hold rich records on every subscriber yet frequently discover churn only after it has occurred, when the customer has already ported their number or cancelled their contract. Retention efforts triggered at that point are largely futile. The core problem is the absence of a forward-looking, per-customer risk signal that would allow intervention while the customer is still reachable and persuadable.

The key issues in the current state are:

Churn is detected reactively rather than predicted proactively, eliminating the window for intervention.

Retention budgets are spent inefficiently because there is no ranking of customers by risk.

Raw customer data is high-dimensional and mixed-type, making manual risk assessment impractical at scale.

The signals that precede churn (contract type, tenure, service gaps) are known individually but are not combined into a single actionable score.

1.4 Proposed System

The proposed system is a supervised classification pipeline wrapped in a service layer. Raw customer attributes are cleaned and transformed into a forty-four-dimensional feature vector, which is passed to a tuned Random Forest classifier that outputs a churn probability. A threshold converts this probability into a binary decision. The trained model is serialised with Joblib and loaded once at service startup, so predictions require no retraining and respond in milliseconds.

The system is composed of four core modules: (1) a data preprocessing and feature-engineering module that mirrors the training-time transformations; (2) an inference module that loads the persisted model and computes probabilities; (3) a FastAPI service layer that validates incoming requests with Pydantic and exposes the `/predict` and `/health` endpoints; and (4) a Bootstrap web interface that collects customer details and displays the result. The technology stack is Python 3.14, scikit-learn and XGBoost for modelling, FastAPI and Uvicorn for serving, and HTML/Bootstrap for the front end.

1.5 Project Scope

Included:

End-to-end pipeline from raw CSV to a live prediction endpoint.

Feature engineering, model comparison, hyperparameter tuning, and explainability.

A validated REST API and an interactive single-page web form.

Model persistence and load-once inference.

Excluded:

Automated model retraining or continuous learning from live traffic.

User authentication, multi-tenancy, and role-based access control.

Integration with a live telecom billing or CRM system.

Limitations:

Predictions are bounded by the ceiling of the source dataset (approximately 81% accuracy, 0.846 AUC).

The model reflects the behaviour captured in a static historical dataset and would require retraining on current data before real deployment.

1.6 Assumptions & Constraints

1.6.1 Assumptions

The input customer attributes are supplied in the same schema and units as the training data.

The historical churn patterns in the dataset are broadly representative of the target customer base.

TotalCharges is missing only for customers with zero tenure and can be safely imputed.

A probability threshold of 0.5 is an acceptable default decision boundary for the demonstration.

1.6.2 Constraints

The model quality is capped by the information content of the public dataset.

Development was constrained to open-source, locally runnable tools with no paid cloud services.

The class imbalance (26.5% churn) constrains raw accuracy as a meaningful metric, requiring AUC as the primary criterion.

1.7 Social Benefits

Enables fairer, needs-based retention offers so that at-risk customers receive support before losing service continuity.

Reduces wasteful mass marketing by targeting only customers who genuinely need engagement.

Improves service quality feedback loops, since the model surfaces the service gaps most associated with dissatisfaction.

Provides a reusable, explainable template that smaller providers can adopt without proprietary tooling.

1.8 Report Layout

Chapter 1 introduces the domain, the churn problem, the aim and objectives, and the scope of the proposed system. Chapter 2 reviews the background statistics of churn and surveys existing tools and methods, identifying the gap this project addresses. Chapter 3 analyses the stakeholders and the functional and non-functional requirements, and specifies the development model and environment. Chapter 4 presents the software design, including the pipeline, system architecture, data representation, and process-flow diagrams. Chapter 5 details the implementation · the core algorithm, external libraries, and the user interface. Chapter 6 describes the testing strategy and reports measured results. Chapter 7 concludes with problems faced, lessons learned, limitations, and future work.

CHAPTER 2: BACKGROUND AND EXISTING WORK

This chapter situates the project within the wider field. It first presents the background statistics that motivate churn prediction, then reviews existing tools, algorithms, and research approaches, and finally synthesises these into an explicit statement of the gap that this project fills.

2.1 Background

Churn prediction is among the most studied applications of supervised learning in business analytics, precisely because the financial stakes are concrete and the data is abundant. In subscription businesses, a small percentage improvement in the monthly retention rate compounds into a large difference in customer lifetime value. The Telco Customer Churn dataset used in this project is a widely adopted benchmark: 7,043 records, nineteen usable predictive attributes after cleaning, and a positive (churn) class prevalence of 26.5%.

The dataset exhibits the class imbalance typical of churn problems · roughly three retained customers for every one who churns. This imbalance is important because a naive classifier that always predicts 'no churn' would achieve 73.5% accuracy while being operationally worthless. Consequently, the field has converged on ranking metrics such as ROC-AUC, which measure a model's ability to order customers by risk independently of any single decision threshold. The pipeline adopted in this project, shown in Figure 1, reflects the standard staged workflow used across the churn-prediction literature.

Figure 1 End-to-end machine learning pipeline adopted in this project

2.2 Literature Review

The following subsections review the principal families of tools and methods relevant to churn prediction and to the serving of machine learning models, noting what each offers and where each falls short for this project's needs.

2.2.1 Logistic Regression

Logistic Regression is the classical baseline for binary churn classification. It is fast to train, produces well-calibrated probabilities, and yields directly interpretable coefficients. Its limitation is that it assumes a linear relationship between the log-odds of churn and the features; it cannot natively capture the interaction effects · such as fiber-optic service combined with the absence of technical support · that are known to drive churn. It therefore serves as a useful lower bound rather than a production candidate.

2.2.2 Random Forest

Random Forest is an ensemble of decision trees trained on bootstrapped samples with random feature subsets. It captures non-linearities and feature interactions automatically, is robust to unscaled and mixed-type inputs, resists overfitting through averaging, and provides feature-importance estimates. Its drawbacks are larger model size and lower interpretability than a single tree. For a tabular, moderate-sized dataset such as this one, Random Forest is a strong and frequently winning choice.

2.2.3 Gradient Boosting (XGBoost / CatBoost)

Gradient-boosted tree libraries such as XGBoost and CatBoost build trees sequentially, each correcting the errors of the last, and often top tabular-data leaderboards. They offer fine-grained regularisation and native handling of missing values. However, they are more sensitive to hyperparameter choices and can overfit small datasets without careful tuning. In this project they were evaluated head-to-head with Random Forest but did not surpass it on ROC-AUC for this particular dataset.

2.2.4 Commercial Churn Platforms

Enterprise CRM and analytics suites (for example, Salesforce Einstein and various telecom-specific retention platforms) offer built-in churn scoring. They provide polished dashboards and integration but are proprietary, costly, opaque in their modelling, and not adaptable at the code level. They are unsuitable for a transparent, self-hosted, educational implementation.

2.2.5 Model-Serving Frameworks

For deployment, options range from Flask (minimal, synchronous), to FastAPI (asynchronous, with automatic validation and OpenAPI docs), to heavyweight platforms such as TensorFlow Serving or BentoML. Flask lacks first-class request validation; the heavyweight platforms add operational complexity disproportionate to a single-model service. FastAPI occupies the sweet spot of low overhead with built-in Pydantic validation and interactive documentation.

2.3 Literature Summary

The review shows that the individual building blocks of churn prediction are mature and well understood: tree-ensemble models dominate tabular classification, ROC-AUC is the accepted metric under class imbalance, and FastAPI is a proven lightweight serving layer. What existing solutions do not provide, in a single package, is a fully transparent, open-source, reproducible pipeline that combines domain-specific feature engineering, rigorous model comparison, explainability, and a working end-to-end service that a non-specialist can run locally.

Commercial platforms deliver serving but hide the model; academic scripts deliver modelling but rarely a deployable interface; tutorials deliver one algorithm but rarely a comparison with tuning and explainability. This project fills that gap by delivering the complete chain · from raw CSV to a validated, explainable, web-served prediction · with every decision documented and every artifact reproducible.

CHAPTER 3: REQUIREMENTS ANALYSIS

This chapter identifies the stakeholders of the system, describes how the requirements were gathered, enumerates the functional and non-functional requirements with a traceability matrix, presents the key use cases, justifies the chosen development model, and specifies the hardware and software environment.

3.1 Stakeholders List (Actors)

Primary Users:

Retention Analyst · enters customer details and interprets the churn score to plan interventions.

End User / Operator · a non-technical staff member who uses the web form to obtain predictions.

Development Team (roles only):

Data Scientist · responsible for EDA, feature engineering, model training, and evaluation.

Backend Developer · responsible for the FastAPI service, validation, and persistence.

Frontend Developer · responsible for the Bootstrap web interface.

Secondary Stakeholders:

Business Owner · consumes aggregate churn insight to allocate retention budget.

3.2 Requirements Elicitation

Requirements were derived from three sources: analysis of the structure and content of the Telco Customer Churn dataset, which fixed the input schema; a review of existing churn tools and academic methods (Chapter 2), which established the expected modelling and serving capabilities; and the practical demands of an end-to-end demonstration, which dictated the need for validation, persistence, and a usable interface.

3.2.1 Functional Requirements

The system shall accept a customer's demographic, service, and account attributes as structured input.
The system shall validate all incoming input against a defined schema and reject malformed requests.
The system shall transform raw input into the engineered forty-four-feature representation used at training time.

The system shall load a pre-trained model without retraining at request time.

The system shall return a binary churn decision and a churn probability for each request.

The system shall expose a health-check endpoint reporting service availability.

The system shall provide a web interface through which a user can submit customer details and view the prediction.

The system shall serve interactive API documentation automatically.

3.2.2 Non-Functional Requirements

Table 1 Non-functional requirements

3.2.3 Requirements Traceability Matrix

Table 2 Requirements traceability matrix

3.3 Use Case Design/Description

Use Case 1: Predict Churn via Web Form

Actors: End User / Retention Analyst

Flow: 1) User opens the web page. 2) User fills in customer attributes. 3) User submits the form. 4) The browser POSTs the data to /predict. 5) The service validates, engineers features, and runs inference. 6) The result (churn label and probability) is displayed.

Outcome: The user sees a churn decision and probability for the entered customer.

Use Case 2: Programmatic Prediction via API

Actors: Backend Developer / External System

Flow: 1) Client sends a JSON POST to /predict. 2) Pydantic validates the payload. 3) Features are engineered. 4) The model returns a probability. 5) A JSON response is returned.

Outcome: The calling system receives a machine-readable churn prediction.

Use Case 3: Health Check

Actors: Monitoring System

Flow: 1) Monitor calls GET /health. 2) Service returns status ok.

Outcome: The monitor confirms the service is live.

3.4 Software Development Life Cycle Model

An incremental, iterative model was adopted, organised into the six phases shown in the pipeline. Each phase produced a working, testable artifact · a cleaned dataset, a trained model, a running API, a web form · that fed the next. This model was chosen over Waterfall because machine learning work is inherently exploratory: the choice of features and algorithm cannot be fully specified up front and must be revised based on measured results. It was chosen over a heavyweight Agile ceremony structure because the project is executed by a small team where formal sprint overhead would add cost without benefit. The incremental approach allowed early phases (EDA, cleaning) to stabilise while later phases (tuning, serving) were still being explored.

3.5 Specific Requirements (Hardware and Software Requirements)

3.5.1 Hardware Requirement

Table 3 Hardware requirements

3.5.2 Software Requirement

Table 4 Software requirements

CHAPTER 4: SOFTWARE DESIGN SPECIFICATION

This chapter presents the design of the system. It describes the overall design approach and work breakdown, the layered system architecture, the data representation through entity, class, and data-flow models, and the dynamic process flow through flowchart, sequence, and activity diagrams.

4.1 Design Models

The system follows a layered, modular design. Responsibilities are separated into a model layer (the persisted classifier and its feature schema), an application layer (validation, feature engineering, and inference within FastAPI), and a client layer (the web form). This separation was chosen because it isolates the

volatile parts of the system · the model, which may be retrained · from the stable serving contract, so the model can be swapped without altering the API or the interface. Development-time concerns are further modularised into independent Jupyter notebooks, one per pipeline stage, keeping exploratory code out of the runtime path.

4.2 Work Breakdown Structure

The project decomposes into four Level-1 work packages · Data, Modeling, Serving, and Documentation · each with Level-2 tasks, as shown in Figure 2.

Figure 2 Work breakdown structure

4.3 System Architecture

The system uses a three-tier client-server architecture. The client tier is a browser rendering a Bootstrap form. The application tier is a FastAPI service that validates requests, engineers features, and performs inference. The model tier consists of the serialised Random Forest and its feature-column definition, loaded into memory at startup. This architecture was chosen over a monolithic notebook because it cleanly separates the trained model from the serving logic, and over a microservices design because a single model does not justify the operational overhead of multiple services.

4.3.1 Block Diagram

Figure 3 shows the high-level blocks and the flow of a request from the browser through validation, feature engineering, and inference, and back as a JSON response.

Figure 3 System block diagram

4.3.2 Component Diagram

Figure 4 shows the development and runtime components and their dependencies: notebooks produce artifacts, and the runtime consumes those artifacts.

Figure 4 Component diagram

4.3.3 Software Architecture Diagram

The software architecture mirrors the block structure and is depicted in the hierarchical view of Figure 5, which maps the physical project layout onto the architectural layers.

Figure 5 Software architecture / project hierarchy

4.4 Data Representation

4.4.1 Entity-Relationship Diagram (ERD)

The domain is modelled around a central Customer entity that is associated one-to-one with its Services, its Account details, and the resulting Prediction, as shown in Figure 6. Although the runtime does not use a relational database, the ERD documents the logical grouping of the input attributes and the derived prediction.

Figure 6 Entity-relationship diagram of the customer domain

4.4.2 UML Class Diagram

Figure 7 shows the principal software classes. CustomerInput is the Pydantic model defining and validating the request schema; PredictionOut defines the response; ChurnAPI holds the endpoint operations; and ModelArtifacts wraps the loaded classifier and feature columns.

Figure 7 UML class diagram

4.4.3 Data Flow Diagram (DFD)

The Level-0 (context) DFD in Figure 8 shows the single external actor exchanging data with the system. The Level-1 DFD in Figure 9 decomposes the system into its four internal processes · validation, feature engineering, inference, and response formatting · together with the model-artifact data store.

Figure 8 Data flow diagram · Level 0 (context)

Figure 9 Data flow diagram · Level 1

4.4.4 Object Interaction (Runtime Class View)

At runtime the ChurnAPI object receives a validated CustomerInput, constructs the feature vector, and delegates scoring to the ModelArtifacts object, which calls predict_proba on the loaded Random Forest. The resulting probability is wrapped in a PredictionOut object and serialised to JSON. This interaction is the object-level realisation of the class diagram in Figure 7.

4.4.5 Hierarchical Diagram

The hierarchical decomposition of the project structure is presented in Figure 5 above, which doubles as the software architecture view.

4.5 Process Flow / Representation

4.5.1 Flowchart

Figure 10 traces the control flow of a single prediction request, including the validation branch that returns an error for a malformed schema and the threshold branch that converts probability into a binary decision.

Prediction Result · Churn Probability Output

Figure 10 Prediction request flowchart

4.5.2 Sequence Diagram

Figure 11 shows the ordered interaction between the user, the web UI, the FastAPI service, the Pydantic validator, and the model during a prediction.

Figure 11 Prediction sequence diagram

4.5.3 Activity Diagram

Figure 12 depicts the overall training-to-serving activity, from loading the dataset through cleaning, encoding, feature engineering, splitting, training, tuning, model selection, and finally serving.

Figure 12 Training-to-serving activity diagram

CHAPTER 5: IMPLEMENTATION

This chapter describes how the design was realised. It presents the main algorithm and its significant sub-algorithms, the external libraries and services used, and the user interface. It also documents each pipeline phase and each project component, explaining what it does, why it was chosen, and why alternatives were rejected.

5.1 Algorithm

The main end-to-end algorithm, covering training through serving, is:

Load the raw Telco Customer Churn CSV into a DataFrame.

Coerce TotalCharges to numeric; impute the eleven missing values (all zero-tenure customers) and drop duplicate records.

Drop non-predictive columns: customerID (identifier), gender and MultipleLines (near-zero mutual information).

One-hot encode categorical attributes with drop_first to avoid the dummy-variable trap.

Engineer fourteen derived features (ratios, counts, and high-risk interaction flags).

Split the data 70/30 into training and test sets with a fixed random seed.

Train Logistic Regression, Random Forest, and XGBoost baselines.

Tune the leading model with GridSearchCV using ROC-AUC as the scoring metric.

Select the tuned Random Forest as the production model and compute SHAP values for interpretability.

Serialise the model and feature-column list with Joblib.

At service startup, load the artifacts once into memory.

For each request: validate, engineer features, call predict_proba, apply the 0.5 threshold, and return the label and probability.

The significant sub-algorithm is the request-time feature engineering, which must exactly reproduce the training-time transformations:

Initialise a zero vector of length 44 indexed by the persisted feature columns.

Populate the four numeric attributes (SeniorCitizen, tenure, MonthlyCharges, TotalCharges).

Compute charge_per_month, tenure_x_monthly, and monthly_minus_avg.

Set binary flags for Partner, Dependents, PhoneService, PaperlessBilling, and the six service indicators.

Aggregate the service flags into services_count, no_services, and has_streaming.

Compute the high-risk interaction flags: fiber_no_support, fiber_no_security, month_to_month_fiber, new_customer, and the ordinal long_contract.

Set the one-hot columns for InternetService, Contract, and PaymentMethod.

Assemble the vector into a single-row DataFrame with the correct column order and score it.

5.2 Pipeline Phases and Components

The following describes each phase of the pipeline, what it does, and the rationale for the tools used.

Phase 1 · Exploratory Data Analysis (01-eda.ipynb)

Establishes the shape of the problem: 7,043 rows, the 26.5% churn rate, data types, missing values, and the relationship between churn and attributes such as contract type and tenure. pandas was used for manipulation and matplotlib/seaborn for visualisation, chosen because they are the de facto standard for tabular EDA and integrate directly with the notebook workflow.

Phase 2 · Preprocessing & Feature Engineering (02-preprocessing.ipynb)

Cleans the data and produces the model-ready feature matrix. One-hot encoding via pandas `get_dummies` with `drop_first` was chosen over label encoding because the categorical variables are nominal, and label encoding would impose a false ordinal relationship. Fourteen engineered features were added to expose interactions that tree models can exploit. The train/test split uses scikit-learn's stratified split to preserve the churn ratio in both partitions.

Phase 3 · Model Training & Tuning (03-model-training.ipynb)

Trains and compares Logistic Regression, Random Forest, and XGBoost. Random Forest was selected because it delivered the best ROC-AUC (0.846) while requiring no feature scaling and resisting overfitting; XGBoost was competitive but did not surpass it on this dataset, and Logistic Regression served as the interpretable baseline. GridSearchCV was chosen for tuning over a manual search for reproducibility and over RandomizedSearchCV because the tuned parameter grid was small enough to search exhaustively. SHAP was used for explainability because it provides theoretically grounded, per-feature attributions.

Phase 4 · Persistence & API (04-api.ipynb, run.py)

Serialises the model with Joblib, chosen over pickle because it is more efficient for the large NumPy arrays inside a Random Forest. The service uses FastAPI with Pydantic validation and Uvicorn as the ASGI server. FastAPI was chosen over Flask for its automatic request validation and OpenAPI documentation, and over heavyweight serving platforms for its minimal footprint. The model is loaded once at startup so predictions incur no I/O.

Phase 5 · Web Interface (templates/index.html)

A single Bootstrap page collects customer attributes and displays the prediction. Bootstrap was chosen over a JavaScript framework such as React because the interface is a single form; a full framework would add a build toolchain and bundle overhead disproportionate to the need.

5.3 External APIs and Libraries

The system is self-contained and does not call third-party network APIs at runtime; all intelligence is in the local model. The significant external libraries are:

scikit-learn: Provides the Random Forest and Logistic Regression estimators, the train/test split, and GridSearchCV. Called during training; the fitted estimator is serialised for serving.

XGBoost: Provides the gradient-boosting candidate evaluated during model selection.

FastAPI + Uvicorn: Provide the REST layer and ASGI server. FastAPI auto-generates interactive docs at `/docs`.

Pydantic: Enforces the request schema at the trust boundary, returning a structured 422 error for invalid input.

Joblib: Serialises and deserialises the trained model and feature-column list.

SHAP: Computes feature attributions for explainability during development.

5.4 User Interface

The frontend is a single responsive HTML page styled with Bootstrap 5. It presents a form covering the customer's demographic, service, and account attributes, with dropdowns for categorical fields and numeric inputs for tenure and charges. On submission it issues a fetch POST to `/predict` and renders the returned churn label and probability, colour-coding the result to distinguish high-risk from low-risk customers. Bootstrap was selected for its zero-build, CDN-based deployment and its mobile-responsive grid, which suited a lightweight single-form interface far better than a component framework would have.

CHAPTER 6: SYSTEM TESTING

This chapter describes how the system was verified. It covers the manual testing performed across system, unit, functional, and integration levels, the approach to automated testing, and the measured results against the requirements.

6.1 Manual Testing

Manual testing was necessary because the system combines a statistical model, a validation layer, and a web interface · categories of behaviour that benefit from direct human inspection of end-to-end outcomes, especially confirming that a plausible customer profile yields a sensible probability.

6.1.1 System Testing

The complete flow was exercised from the browser: filling the form, submitting, and confirming that a churn label and probability were displayed. A high-risk profile (month-to-month contract, fiber optic, no technical support, short tenure) was confirmed to return a high churn probability, and a low-risk profile (two-year

contract, long tenure) a low probability, validating that the end-to-end system behaves consistently with domain expectations.

6.1.2 Unit Testing

Individual components were tested in isolation. Example cases:

Table 5 Sample unit test cases

6.1.3 Functional Testing

Each functional requirement was checked against a concrete scenario. For example, the input-validation requirement was tested by POSTing a payload with a missing field: the service correctly returned HTTP 422 with a descriptive error rather than crashing. The prediction requirement was tested by POSTing a complete valid payload and confirming a 200 response containing both churn and probability, for instance {"churn": 1, "probability": 0.6153}.

6.1.4 Integration Testing

Three integration boundaries were exercised. The Frontend-Backend integration was verified by confirming the web form's fetch call reached /predict and rendered the response. The Backend-Model integration was verified by confirming the loaded artifacts produced predictions matching those from the training notebook for identical inputs. The earlier one-hot mapping defects (InternetService, PaymentMethod) were found and fixed during this stage, when notebook-trained predictions were compared against the served predictions.

6.2 Automated Testing

6.2.1 Tool Used

Automated tests use pytest with the FastAPI TestClient for API-level tests and Playwright for browser-based end-to-end UI tests. The API test suite (test_api.py) covers health checks, valid predictions, input validation (wrong types and missing fields), feature-vector integrity, and output consistency. The UI test (test_ui.py) uses Playwright to launch a headless Chromium browser, navigate to the web form, fill customer details, submit, and verify the result card displays a churn decision with probability. Representative test cases:

```
import requests
r = requests.post('http://localhost:8000/predict', json=payload)
assert r.status_code == 200
body = r.json()
assert body['churn'] in (0, 1)
assert 0.0 <= body['probability'] <= 1.0
```

6.3 Results & Discussion

The model achieved a ROC-AUC of 0.846 and an accuracy of approximately 81% on the held-out test set. All six API tests passed (test_api.py) and the Playwright browser UI test passed (test_ui.py), confirming the service layer, validation, and web form function correctly. Under the dataset and an accuracy of approximately 81% on the held-out test set. Under the dataset's 26.5% churn prevalence, this AUC represents strong ranking performance - the model reliably orders customers by risk, which is the operationally important property. The served /predict endpoint returned predictions in well under the 50 ms non-functional target (raw inference measured at roughly 19 ms median), and the /health endpoint and web form both returned successful responses. All functional requirements were met. The accuracy plateau near 81% is attributable to the information ceiling of the source dataset rather than model choice, a conclusion confirmed by the fact that XGBoost, CatBoost, and stacked ensembles did not exceed it.

CHAPTER 7: CONCLUSION

This chapter reflects on the project: the problems encountered and how they were resolved, the extent to which the objectives were met, the current limitations, and the avenues for future work.

7.1 Problems Faced and Lessons Learned

Problems Faced:

The TotalCharges column contained blank strings for zero-tenure customers, causing type errors on load.

One-hot column names produced at training time did not initially match the names constructed at request time, yielding wrong predictions.

HTTP round-trip latency dominated end-to-end timing even though raw inference was fast.

Accuracy stalled near 81% despite trying multiple algorithms and ensembles.

How They Were Resolved:

Blank TotalCharges values were coerced to numeric and imputed, and duplicates were removed.
The one-hot mapping was corrected by aligning the request-time column names exactly to the persisted feature-column list.
The model was loaded once at startup and inference kept lightweight; the latency was traced to connection setup rather than the model.
Systematic experiments confirmed the plateau was a dataset ceiling, so effort was redirected to robust feature engineering and clean serving rather than chasing unattainable accuracy.
Lessons Learned:
We learned that reproducing training-time transformations exactly at serving time is as important as the model itself.
We learned to trust AUC over raw accuracy when the classes are imbalanced.
We learned that knowing when a dataset has reached its ceiling prevents wasted effort.

7.2 Conclusion

The project achieved its aim: a working machine learning system that predicts telecom customer churn and serves those predictions through a validated API and an interactive web interface. Mapping back to the objectives of Section 1.2, all were met · EDA was completed, the data was cleaned, fourteen features were engineered, three algorithms were compared, the leading model was tuned, its behaviour was explained with SHAP, the model was persisted and served through FastAPI with Pydantic validation, and a Bootstrap web form was delivered. The tuned Random Forest reached a ROC-AUC of 0.846, and the end-to-end system meets its performance and usability requirements.

7.3 Limitations of the Project

Predictive performance is bounded by the information ceiling of the static public dataset.
The system does not retrain automatically and would need refreshing on live data before real deployment.
There is no authentication, logging, or persistence of predictions in the current version.
The decision threshold is fixed at 0.5 rather than tuned to a business cost matrix.

7.4 Future Work

Add SQLite or a database layer to log every prediction for monitoring and drift detection.
Introduce cost-sensitive threshold tuning aligned to the real cost of retention versus loss.
Containerise the service with Docker and add a CI pipeline for automated testing and deployment.
Add an automated retraining schedule triggered by data-drift detection.
Enrich the dataset with behavioural and usage signals to push past the current accuracy ceiling.

REFERENCES

IBM, "Telco Customer Churn" dataset, IBM Cognos Analytics sample data. Available: <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>
L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.
T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD, 2016, pp. 785-794.
F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Advances in Neural Information Processing Systems 30, 2017.
S. Ramirez, "FastAPI Documentation." Available: <https://fastapi.tiangolo.com>
API Documentation · Swagger UI at /docs
POST /predict Endpoint · Request Schema
The pandas development team, "pandas: powerful Python data analysis toolkit." Available: <https://pandas.pydata.org>

PSEUDO CODE

Algorithm 1: Predict Customer Churn
Algorithm PredictChurn
Input: customer (validated attribute record)
Output: churn (0 or 1)
probability (float in [0,1])
BEGIN

```

x <- zero vector of length 44 indexed by feature_cols
x[numeric] <- SeniorCitizen, tenure, MonthlyCharges, TotalCharges
x['charge_per_month'] <- TotalCharges / (tenure + 1)
x['tenure_x_monthly'] <- tenure * MonthlyCharges
x['monthly_minus_avg'] <- MonthlyCharges - charge_per_month
FOR EACH service IN service_list DO
x[service + '_bin'] <- 1 IF customer[service] = 'Yes' ELSE 0
END FOR
x['services_count'] <- sum of service bins
IF InternetService = 'Fiber optic' AND TechSupport = 'No' THEN
x['fiber_no_support'] <- 1
END IF
set one-hot columns for InternetService, Contract, PaymentMethod
p <- model.predict_proba(x)[1]
churn <- 1 IF p >= 0.5 ELSE 0
RETURN churn, round(p, 4)
END
Algorithm 2: Train and Select Model
Algorithm TrainSelect
Input: dataset D
Output: best_model, feature_cols
BEGIN
D <- clean(D); D <- engineer_features(D)
X, y <- split_features_target(D)
Xtr, Xte, ytr, yte <- train_test_split(X, y, 0.7, stratify=y)
best_auc <- 0
FOR EACH algo IN {LogReg, RandomForest, XGBoost} DO
m <- fit(algo, Xtr, ytr)
auc <- roc_auc(m, Xte, yte)
IF auc > best_auc THEN best_model <- m; best_auc <- auc END IF
END FOR
best_model <- GridSearchCV(best_model, grid, scoring='roc_auc')
persist(best_model, feature_cols)
RETURN best_model, feature_cols
END

```

APPENDICES

Appendix A · Project Structure

```

customer-churn-prediction/
data/ raw CSV + train/test parquet splits
models/ churn_model.pkl, feature_cols.pkl
notebooks/ 01-eda, 02-preprocessing, 03-training, 04-api
templates/ index.html (Bootstrap web form)
run.py FastAPI service entry point
start.bat launcher script
pyproject.toml

```

Appendix B · Engineered Features

Table 6 Engineered features

Appendix C · Sample API Request/Response

```

POST /predict
{
"SeniorCitizen": 0, "Partner": "Yes", "Dependents": "No",
"tenure": 12, "PhoneService": "Yes",
"InternetService": "Fiber optic", "Contract": "Month-to-month",
"PaymentMethod": "Electronic check",

```

```
"MonthlyCharges": 70.0, "TotalCharges": 840.0, ...
}
-> 200 OK
{"churn": 1, "probability": 0.6153 }
```

LIST OF ABBREVIATIONS

MACHINE LEARNING PIPELINE

This section presents the complete machine learning pipeline as an integrated whole, describing each stage, the data that flows between stages, and the transformation performed at each step. The pipeline is the backbone of the system: every prediction served at runtime is the product of the same staged process that produced the model during development.

Figure 13 End-to-end machine learning pipeline

Stage 1 · Data Ingestion

The raw Telco Customer Churn CSV (7,043 rows) is loaded into a pandas DataFrame. This stage fixes the input contract for every downstream step; all later transformations assume the column names and types established here.

Stage 2 · Data Cleaning

The TotalCharges column is coerced to numeric, the eleven blank values (all zero-tenure customers) are imputed, and duplicate records are removed. Non-predictive columns · customerID, gender, and MultipleLines · are dropped, the latter two because their mutual information with churn is effectively zero.

Stage 3 · Feature Engineering

Categorical attributes are one-hot encoded with drop_first, and fourteen derived features are added · ratios (charge_per_month), aggregates (services_count), and high-risk interaction flags (fiber_no_support). This raises the feature space from the raw attributes to forty-four inputs and is the single most impactful lever on model quality after algorithm choice.

Stage 4 · Train/Test Split

The engineered matrix is split 70/30 with stratification on the churn label and a fixed random seed, preserving the 26.5% churn ratio in both partitions and guaranteeing reproducibility.

Stage 5 · Model Training

Logistic Regression, Random Forest, and XGBoost are trained on the training partition and evaluated on the held-out test partition using ROC-AUC as the primary criterion under class imbalance.

Stage 6 · Hyperparameter Tuning

The leading model is refined with GridSearchCV over a cross-validated parameter grid, again scored on ROC-AUC, yielding the tuned Random Forest with an AUC of 0.846.

Stage 7 · Model Persistence

The tuned model and the ordered feature-column list are serialised with Joblib into the models directory, decoupling training from serving so the runtime never retrains.

Stage 8 · Serving

The FastAPI service loads the artifacts once at startup and exposes the /predict endpoint. At request time it reproduces the exact feature-engineering transformations from Stage 3 before calling the model · ensuring training-serving parity · and returns a churn label and probability, consumed by the Bootstrap web interface.

PROJECT STRUCTURE

This section documents the physical organisation of the project on disk and the role of each directory and file. The layout separates development-time notebooks, persisted artifacts, and runtime serving code, so that the exploratory work stays out of the production path while remaining fully reproducible. The hierarchy is shown in Figure below.

Figure 14 Project directory hierarchy

The directory tree is as follows:

```

customer-churn-prediction/
|
+-- data/
| +-- telco_customer_churn.csv raw source dataset (7043 rows)
| +-- X_train.parquet training features (70%)
| +-- X_test.parquet test features (30%)
| +-- y_train.parquet training labels
| +-- y_test.parquet test labels
|
+-- models/
| +-- churn_model.pkl tuned Random Forest (44 features)
| +-- feature_cols.pkl ordered feature-column list
| +-- scaler.pkl legacy StandardScaler (unused by RF)
|
+-- notebooks/
| +-- 01-eda.ipynb exploratory data analysis
| +-- 02-preprocessing-...ipynb cleaning + feature engineering
| +-- 03-model-training-...ipynb training, tuning, SHAP
| +-- 04-api.ipynb API endpoint development
|
+-- templates/
| +-- index.html Bootstrap web form
|
+-- run.py FastAPI service entry point
+-- start.bat double-click launcher
+-- pyproject.toml dependencies (uv-managed)
+-- build-plan.md phase checklist

```

The responsibilities of each component are summarised below.

Table 7 Project component responsibilities

This structure was chosen because it enforces a clean separation between the three lifecycles present in a machine learning product: the exploratory lifecycle (notebooks), the artifact lifecycle (models and data), and the serving lifecycle (run.py and templates). A change to any one · retraining the model, revising a notebook, or altering the API · can be made without disturbing the others, which is the practical benefit of the layered design described in Chapter 4.